

RELAZIONE VERIFICA 19/06/2026

```
1  import datetime
2
3  while True
4      comando_utente = input("Cosa vuoi sapere? ")
5      if comando_utente == "esci":
6          print("Arrivederci!")
7          break
8      else:
9          print(assistente_virtuale(comando_utente))
10
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.datetime.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
23
```

Capire cosa fa il programma senza eseguirlo:

Il codice è assistente virtuale a riga di comando: chiede all'utente di scrivere una domanda, e se riconosce una delle tre frasi previste ("Qual è la data di oggi?", "Che ore sono?", "Come ti chiami?") risponde di conseguenza con data, ora o nome. Se la domanda non corrisponde a nessuna di queste, risponde che non ha capito. Il programma continua a chiedere input finché l'utente non scrive "esci", a quel punto saluta e termina.

Individuare nel codice sorgente le casistiche non standard che il programma non gestisce (esempio, comportamenti potenziali che non sono stati contemplati):

1. Sensibilità a maiuscole/minuscole:

Se l'utente scrive "qual è la data di oggi?" (minuscolo) o "QUAL È LA DATA DI OGGI?", il confronto fallisce e l'assistente risponde "Non ho capito", anche se la domanda è sostanzialmente identica. Quindi di conseguenza una soluzione sarebbe aggiungere `".lower()"` alla fine della stringa.

2. Sensibilità a spazi e punteggiatura:

Una domanda come "Qual è la data di oggi" (senza punto interrogativo finale) o con spazi doppi "Qual è la data di oggi?" non corrisponde a nessuna delle frasi previste. Il confronto è rigido, per ovviare a questo dopo la stringa aggiungere ".strip()". Di conseguenza aggiungendolo al "problema" precedente verrebbe: `input(stringa).strip().lower()`

3. Nessuna gestione di varianti linguistiche o sinonimi:

Frase equivalenti come "Che giorno è oggi?", "Dimmi l'ora", "Chi sei?" non sono previste, anche se semanticamente identiche a quelle gestite. L'assistente non fa nessuna analisi del linguaggio, solo match esatto, la soluzione sarebbe cambiare "==" in "in" e aggiungere le parole equivalenti con "or" di conseguenza scrivere ad esempio: (if "data" in comando or "giorno" in comando:) così che "Che giorno è oggi?" o "Dimmi la data" vengono entrambe riconosciute, perché contengono la parola chiave "data".

4. La condizione "esci" ha lo stesso problema di rigidità

Solo la parola esatta "esci" termina il programma. Scrivere "Esci", "ESCI", "esci per favore", "stop" ecc... non funziona: il programma continuerà a chiedere input all'infinito, dando sempre "Non ho capito la tua domanda." Quindi anche in questo caso sostituire "==" con "in" e di conseguenza scrivere all'interno nel ciclo while True:

if comando_utente in ["esci", "stop", "finito"].

Individuare eventuali errori di sintassi / logici e proporre una soluzione per ognuno di essi:

1. Nel "while True" mancano i due punti:

```
3 while True
```

2. La funzione "assistente_virtuale" è definita DOPO essere stata usata:

In Python le funzioni devono essere definite prima di essere chiamate. Bisogna spostare tutta la `def assistente_virtuale(comando)`, il ciclo while va spostato dopo la costante, in questo modo:

Prima questo:

```
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.datetime.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
```

Poi questo:

```
3 while True
4     comando_utente = input("Cosa vuoi sapere? ")
5     if comando_utente == "esci":
6         print("Arrivederci!")
7     break
8     else:
9         print(assistente_virtuale(comando_utente))
```

3. Intestazione sbagliata del break:

Il break è allineato con if/else, ma deve stare *dentro* il blocco if, altrimenti il ciclo si interrompe sempre (anche quando non scrivi "esci"):

```
if comando_utente == "esci":
    print("Arrivederci!")
break
else:
    print(assistente_virtuale(comando_utente))
```

Quindi spostare con uno spazio il break verso destra.

4. datetime.datetoday() non esiste Il metodo corretto è:

datetime.date.today().

Il codice corretto con l'aggiunta delle casistiche sopra citate:

```
verifica.py > ...
1  import datetime
2
3  def assistente_virtuale(comando):
4      comando = comando.strip().lower()
5
6      if "data" in comando or "giorno" in comando:
7          oggi = datetime.date.today()
8          risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
9      elif "ora" in comando or "ore" in comando:
10         ora_attuale = datetime.datetime.now().time()
11         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
12     elif "chiami" in comando or "chi sei" in comando:
13         risposta = "Mi chiamo Assistente Virtuale"
14     else:
15         risposta = "Non ho capito la tua domanda."
16     return risposta
17
18 while True:
19     comando_utente = input("Cosa vuoi sapere? ").strip().lower()
20     if comando_utente in ["esci", "stop", "quit", "fine"]:
21         print("Arrivederci!")
22         break
23     else:
24         print(assistente_virtuale(comando_utente))
```

Esecuzione del chat-bot con tutte le casistiche da me aggiunte e errori sistemati:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
/usr/bin/python "/home/kali/Desktop/Vs Code/lezione3.py/verifica.py"
(kali@kali)-[~/Desktop/Vs Code/lezione3.py]
$ /usr/bin/python "/home/kali/Desktop/Vs Code/lezione3.py/verifica.py"
Cosa vuoi sapere? come ti chiami
Mi chiamo Assistente Virtuale
Cosa vuoi sapere? che giorno siamo
La data di oggi è 19/06/2026
Cosa vuoi sapere? adesso che ora è
L'ora attuale è 13:26
Cosa vuoi sapere? eScI
Arrivederci!

(kali@kali)-[~/Desktop/Vs Code/lezione3.py]
$
```

Questo dimostra come il mio assistente rispetto al primo codice comprende anche gli equivalenti, che grazie all'inserimento di "in-or" e di ".strip()" e ".lower()" (come mostrato in precedenza), fanno in modo che il mio chatbot, non vada in "tilt" qualora non si digitasse l'esatta stringa.

